SECTION 6 · CHEAT SHEET

Next.js and skills

Every prompt, command, shortcut and setting from this section, lesson by lesson. Prompts are word for word, as typed on screen.

6.1 Section 6 introduction

Concept lesson: nothing to type, run or set.

6.2 Move the site to Next.js

PROMPT

```
/plan Move the cafe website in site/ to Next.js. First make sure Node.js is
installed and on the current LTS version. Every page, the cart and the demo
checkout must look and work exactly as they do now, with the same menu data and
photos. Update scripts/test_order_flow.py so it tests the new site, and it must
pass. Point the menu card's QR code at the new menu page.
```

A new conversation in the cafe project; /plan makes the agent write a plan before it changes anything. It checked Node.js (v24.15.0), then stopped with no plan.

PROMPT

```
You stopped after checking the versions. Is v24.15.0 the current LTS? If not,
upgrade Node.js first, then write the plan.
```

It finds v26 is the current LTS and upgrades with winget. The installer runs once, so **allow this time**; Windows asks for admin rights: Yes.

PROMPT

```
The Node.js installer finished after I clicked Yes on the Windows admin prompt.
Check the version again, then write the plan for the move to Next.js.
```

v26.13.0. It reads every page and script, then writes the plan: one route per page, the photos and the menu data at the same addresses.

PROMPT

```
One change: use the current Next.js version, not 15. Everything else looks good, go
ahead with the migration.
```

The plan's package list said Next.js 15: read a plan for versions. It settles on 16 and migrates until all nine order checks pass.

PROMPT

```
Yes to the cleanup: remove the old HTML pages, site/js and site/css, and any script
that only built them, so there's one site. Then run the order test again and start
the dev server. Don't commit.
```

Its own cleanup offer, accepted. Commands that repeat (build, dev server, order test) get **always allow in this conversation**. The site runs at localhost:3000.

6.2 Move the site to Next.js, the cart error

RESULT

localhost:3000/cart > the Next.js issue badge

A flushSync console error that the order test never sees. Copy it.

PROMPT

```
The cart page shows this error in the browser. Fix it, run the order test again,
and keep the dev server running.
```

The copied error pasted underneath. Flash fixes one flushSync call; the rest stay.

PROMPT

```
Stop here. I'm switching to a stronger model for this fix.
```

Typed as the **No** answer on its next card.

SETTING

Model picker › Gemini 4 Argon Medium

A stronger model for one tricky bug, in this conversation. Back to Flash afterwards.

PROMPT

```
Flash fixed one flushSync call but the cart page still uses flushSync in other
places. Find the real cause of the error and fix it properly, so the cart page
shows no console errors in the browser and the order test still passes. Check both,
and keep the dev server running.
```

Argon removes every flushSync call, fixes hydration, the footer and the favicon: **0 console errors**.

6.3 Skills, Rules or MCP

Concept lesson: nothing to type, run or set.

6.4 Anatomy of a skill

PROMPT

```
Create a workspace skill for our weekly menu update, in .agents/skills/menu-import.
For now only its SKILL.md: a name, a description that says what the skill does and
when to use it, and short steps: read the new menu from
source/KGP_Vibes_Cafe_Menu.pdf, update baseline/menu.csv, sync the website menu,
regenerate the menu card and run the order test. Don't run anything and don't
change the site yet.
```

Flash reads Antigravity's own guide on skills first, then writes one file. Nothing run, no cards.

FILE

.agents/skills/menu-import/SKILL.md

Header: **name** (optional, else the folder name) and **description** (required); then the steps.

RESULT

The description decides

Until a request matches, the agent sees only names and descriptions. Say what it does and when.

SETTING

Customizations › Installed › Skills & Rules

Built-in and global skills only. A workspace skill belongs to its project and is not listed here.

COMMAND

```
/menu
```

Starts a skill by hand. It finds menu-import only in a new conversation: skills load at the start.

6.5 A skill with scripts

PROMPT

```
/menu-import This week's menu is in source/KGP_Vibes_Cafe_Menu.pdf. Give this skill its own script, .agents/skills/menu-import/scripts/import_menu.py, that reads the PDF and updates baseline/menu.csv: match items by name, keep each item's id and the columns the PDF doesn't have, and never add an item twice. Update SKILL.md to use the script, then run every step of the skill. The menu page's category buttons must come from menu.csv. Finally, run the import a second time and show that nothing changes.
```

The chip loads the skill by name. The first turn ended after 11 s with no work.

PROMPT

```
You stopped after listing the files. Please continue with the whole task.
```

A short message to continue is all it needs.

RESULT

Allow run test_order_flow.py?

The skill runs this test every time: **always allow in this conversation**.

PROMPT

```
The import also rewrote eight descriptions in menu.csv with the PDF's wording: commas became long dashes and cafe became café. Keep the descriptions we already have. The import should only change names, prices and categories, and add or remove items. Fix the script, restore those descriptions, run the skill again, then run the import twice.
```

Everything had passed, but nine items were updated and only two had new prices. Read what a run changed, not just that it passed.

FILE

.agents/skills/menu-import/scripts/import_menu.py

Matches by name, keeps ids and the other columns, changes only names, prices and categories. SKILL.md says when and in what order; the script does the exact work the same way every week.

RESULT

git diff baseline/menu.csv

Two lines: Tech Latte $3.75 and Talkie Chai $2.25, every description as it was. The next two imports changed nothing.

RESULT

localhost:3000/menu

The buttons now come from menu.csv: Coffee, Snacks & Meals, Meals, Desserts & Drinks.

6.6 A skill with examples

PROMPT

```
Create a workspace skill called new-page in .agents/skills/new-page, for adding a new page to the cafe website in the house style. Copy site/app/about/page.jsx into its examples folder as the reference page. Add a short checklist to its resources folder: the header and footer come from the layout, colours and fonts from DESIGN.md, prices in dollars with two decimals, a link in the top menu, every image the page uses exists, and the order test passes. The steps in SKILL.md should point to both. Don't create any page yet.
```

Two new files and one copy, no page yet.

FOLDER

.agents › skills › new-page

examples/: work to copy. **resources/**: a checklist. Read only when the skill runs.

PROMPT

```
/new-page Add a Daily Specials page at /specials: one special for each day of the week, Monday to Sunday. Pick items from the menu data that have a photo in site/public/images/menu, and show each one's photo, description and price from the menu data, with an Add to Order button that works like the one on the menu page. Add the page to the top menu.
```

In a new conversation, so the skill is loaded. The first thing Flash opens is the skill.

RESULT

Allow run order flow test suite?

In the sandbox the test can't reach its server: **always allow in this conversation**.

RESULT

localhost:3000/specials

Seven specials, Monday to Sunday, plus an unasked Today badge. Nine checks pass; Add to Order works.

6.7 Global skills

PROMPT

```
Create a global skill called page-check. Put it in the global skills folder, ~/.gemini/config/skills/page-check, not in this project, so every project on this computer can use it. The description should say to use it whenever the user asks to check a web page before it goes live. The steps: find the project's pages, and for each page check the page title, a meta description, one main heading, alt text on every image, that every image file exists, that every link goes somewhere real, and a mobile viewport tag. Then show a pass or fail table for each page, fix only what failed, and say what changed. Don't run the skill now.
```

Sent from the cafe project.

RESULT

Allow write access to this path?

Outside the project, so it asks. Our own folder: **always allow in this conversation**.

FILE

~/.gemini/config/skills/page-check/SKILL.md

Same format, another folder: every project sees it. Customizations > Installed lists it as **Global**.

PROMPT

```
/page-check Check index.html before it goes live.
```

A new conversation in my-website, which has no skills folder. Typing /page shows the global skill.

RESULT

Six pass, one fail

Book a Call and Resume went to missing sections; only those two links changed, to email links.

6.8 Getting auto-discovery right

PROMPT

```
Create a workspace skill called menu-item, for adding one new item to the cafe menu. For now, give it only this description: "Menu helper." The steps: add the item to baseline/menu.csv and site/data/menu.json with the next free id in its category, write the price in USD with two decimals, and if the item has no photo in site/public/images/menu, show a clean placeholder on its card instead of a broken image. Then build the site and run the order test. Don't add any item yet.
```

A skill with six steps and a vague description.

PROMPT

```
Add a Pumpkin Spice Latte to the menu: Coffee, $4.25, espresso with pumpkin spice and steamed milk, vegetarian.
```

A new conversation, no slash. Flash opens menu-item first: the name alone matched. Then its turn ended.

PROMPT

```
Go ahead and add it.
```

It also reads menu-import, follows parts of both and rebuilds the printed menu cards: about 8 minutes. Build and test get **always allow in this conversation**.

PROMPT

```
Two skills in this project overlap: menu-item says only "Menu helper.", and menu-import also claims single menu items. Rewrite just their descriptions so the agent can tell them apart. menu-item: it adds, changes or removes one item on the cafe menu (menu data, price format, photo or placeholder, build, order test); use it whenever the user asks to add, change or remove a single menu item. menu-import: it imports a whole new weekly menu from source/KGP_Vibes_Cafe_Menu.pdf; use it only when a new weekly menu PDF arrives, never for a single item. Change nothing else.
```

A new conversation. A good description says what, when, and when not, if another skill is close.

PROMPT

```
Add a Masala Chai Latte to the menu: Coffee, $3.95, spiced chai with a shot of espresso, vegetarian.
```

The next new conversation picks up both descriptions: menu-item only, about 4 minutes, one file changed.

PROMPT

```
On the menu page, the Pumpkin Spice Latte and Masala Chai Latte cards show a broken image with the alt text over it, not a placeholder. Step 4 of the menu-item skill asks for a clean placeholder. Fix the menu card so any item without a photo shows a clean placeholder, check it on the page, and tell me how you checked.
```

Both runs claimed the placeholder; the page had broken images. Check the result on the page.

RESULT

localhost:3000/menu

Checked in headless Chrome: 12 items with photos, 29 with the clean placeholder, no missing images. The order test passes.